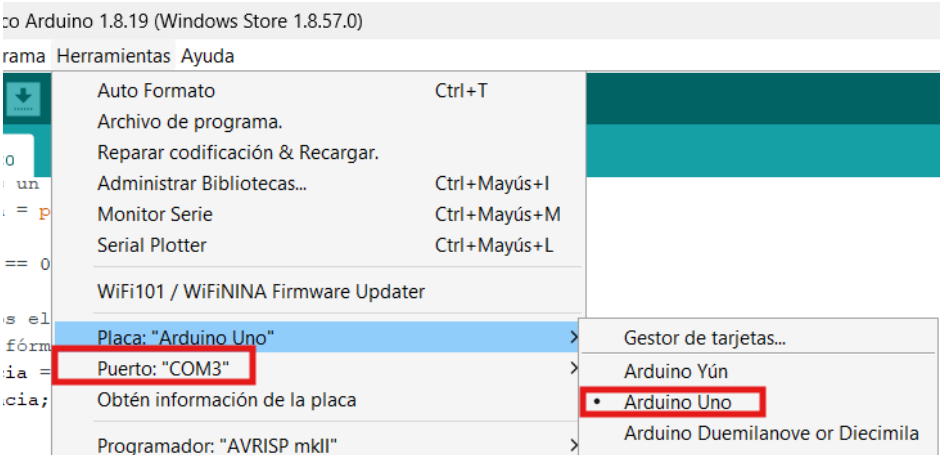
Sistema de Monitoreo de Tanque

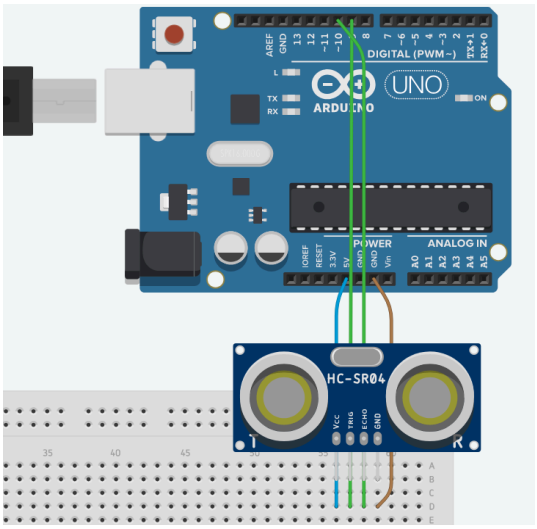
Derly Lorena Novoa Herrera 160005022
Astrid Daniela Martinez Ruiz (160005018)
Andrés Santiago Chavarro Pardo (160004913)
Duvan Felipe Vaquero Cerquera (160005002)

1. Arquitectura Ciber-física (Hardware)

Configuración del Entorno (Despliegue): Para la ejecución del nodo físico, se requiere la instalación del *IDE de Arduino*. La comunicación se establece vía puerto Serial (Baudios: 9600) garantizando la transferencia de datos en tiempo real.



Representación de Conexión Sensor Ultrasónico al ARDUINO



- **Explicación:** Se utiliza una topología de conexión directa. El sensor ultrasónico (HC-SR04) toma alimentación de los 5V del Arduino. Los pines de control se asignaron al bloque digital (TRIG al Pin 9, ECHO al Pin 10) para separar la lógica de sensado de los pines de comunicación serial (0 y 1).

Etapa de Actuación y Aislamiento de Potencia:

Para el control del flujo hídrico, se integra un módulo relé electromecánico de 1 canal (5V) comandado por el Pin Digital 13. Este componente actúa como aislante, separando el circuito lógico del microcontrolador de la carga inductiva de la mini-bomba DC. El motor se conecta en serie utilizando el borne Normalmente Abierto (NO) del relé, garantizando que permanezca desenergizado por defecto y mitigando el riesgo de daño al microprocesador por picos transitorios de corriente.

2. Desglose de Contribuciones y Análisis de Software

Astrid Martinez (Arquitectura de Dominio y Refactorización)

A. Responsabilidad: Limpieza arquitectónica y diseño de Entidades Puras.

B. Implementación de Código: Refactorización y Arquitectura

Análisis del Código y Estructura: Se realizó una reestructuración integral del paquete de **Dominio**, priorizando la eliminación de dependencias circulares y la correcta ubicación de responsabilidades. Anteriormente, el dominio estaba "contaminado" con lógica de orquestación; tras la refactorización, se ha establecido una jerarquía clara donde el Dominio es el núcleo independiente y los Servicios actúan como mediadores.

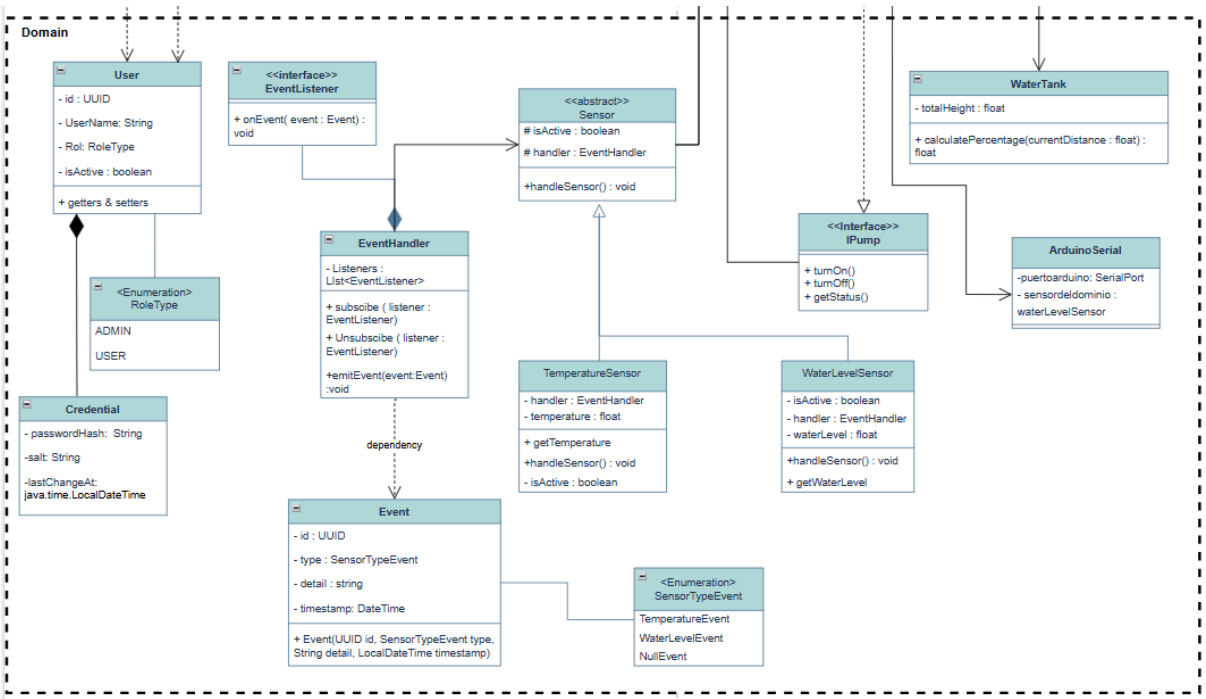


Figura 3. *Conjunto de clases de dominio del sistema WaterTank.*

Refactorización de Clases:

- *IPump:* Actúa como una **abstracción** de la bomba física **(pump)** dentro del **Dominio**. Nos permite indicarle a la bomba **qué** debe hacer, sin importarle los detalles técnicos de **cómo** lo hace. Esta separación garantiza que nuestro código central sea independiente del hardware; si mañana cambiamos el modelo de la bomba o el microcontrolador, el Dominio seguirá funcionando exactamente igual sin modificar una sola línea. Facilitando el intercambio de componentes o el uso de **mocks** para **pruebas unitarias.**

- ***WaterTank (Entidad Pura):*** La clase **WaterTank** ha sido despojada de cualquier acoplamiento con el hardware o servicios externos. Ahora funciona como una Entidad Pura que garantiza el encapsulamiento absoluto, operando exclusivamente sobre sus atributos internos (como capacity). Al limitar sus responsabilidades al cálculo de estado **calculateWaterVolumePercentage()**, lo cual cumple rigurosamente con el principio **GRASP de Alta Cohesión.**

- ***Sensor (Abstracción de Hardware):*** Se implementó **Sensor** como una **clase abstracta,** definiendo un contrato uniforme para cualquier dispositivo de lectura. Esto permite que el sistema sea extensible: **el dominio solo conoce la abstracción "Sensor",** permitiendo integrar sensores de temperatura o nivel sin alterar la lógica de negocio existente.

- ***TemperatureManager y WaterLevelManager (Clases Activas):*** La lógica de control y supervisión se desplazó a la capa de Service. Clases como **TemperatureManager** se definieron como **Clases Activas** (implementando Runnable), encargándose de la ejecución de **hilos independientes** que monitorean y procesan eventos de forma asíncrona. Esto asegura que la entidad **WaterTank** no tenga que "preocuparse" por el tiempo de ejecución o el estado del hilo, **delegando esa carga operativa** a los servicios especializados.

C. Justificación Técnica:

- ***GRASP (Indirección y Fabricación Pura):*** Se introdujo el **EventHandler** como un mecanismo de **indirección.** El dominio emite eventos, pero no sabe quién los escucha, lo que **reduce el acoplamiento** y permite que múltiples servicios (como los Managers) reaccionen a cambios en el tanque de forma independiente.

- ***SOLID (Inversión de Dependencias):*** Los **servicios de alto nivel no dependen de las clases de bajo nivel** (sensores específicos), **sino de abstracciones.** Esto se evidencia en la estructura de directorios, donde el paquete Domain es el centro y no tiene flechas de dependencia apuntando hacia afuera (hacia Service o Infrastructure).

- ***Encapsulamiento y Protección de Variaciones:*** Al mover los métodos mutables y la lógica de control a los Managers en la capa de servicio, protegemos a la entidad **WaterTank** de cambios frecuentes en los requisitos de control (por ejemplo, si cambia el umbral de temperatura para activar la bomba).

D. Evidencia de colaboración en el repositorio de github:

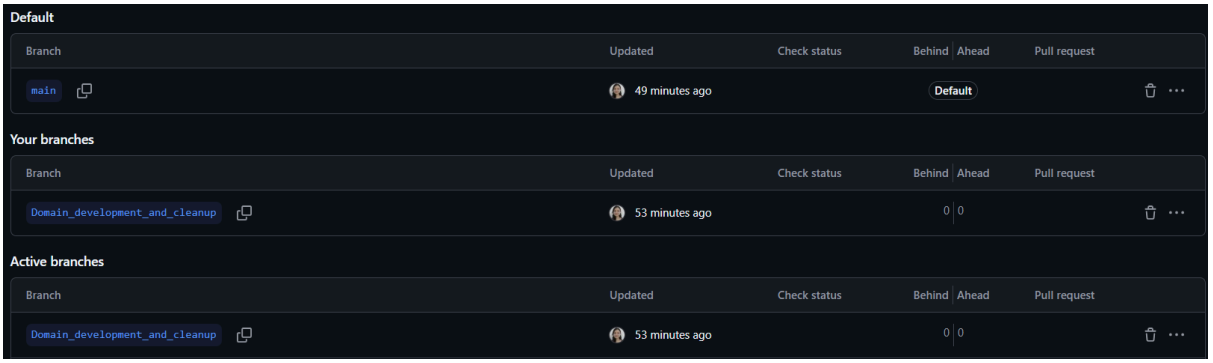


Figura 4. *Flujo de trabajo en git hub (Rama local y principal main).*

En la **Figura 4** se documenta el flujo de trabajo profesional en Git, iniciando con la gestión de la rama local **Domain_development_and_cleanup** para una refactorización segura.

Tras asegurar los cambios, se realizó la **sincronización con el repositorio remoto** en GitHub mediante **push**, garantizando la disponibilidad del código para el equipo. Finalmente, tras verificar la **paridad con pull,** se ejecutó el **merge exitoso hacia la rama main,** consolidando la nueva arquitectura del sistema sin conflictos.

Juan Romero (Adquisición de Datos y Servicios Ciber-Físicos)

- **A. Componente de Hardware:** Programación del Microcontrolador y Sensor Ultrasónico.

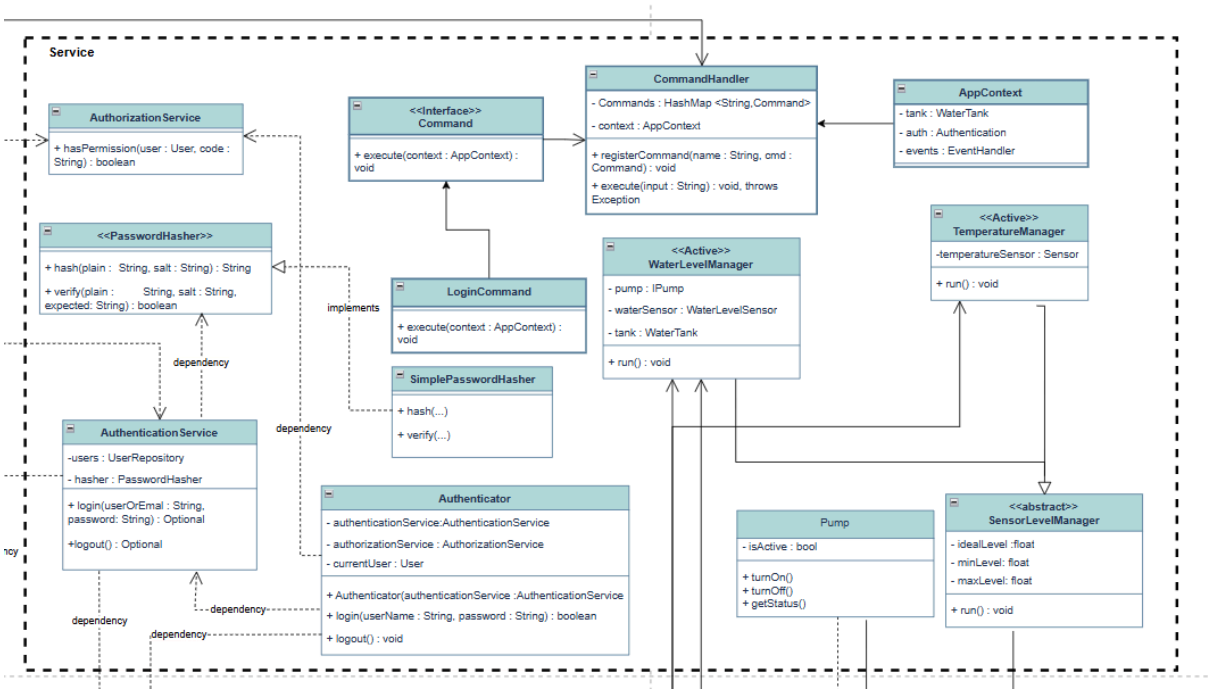


Figura 5. *Conjunto de clases de la capa de servicio del sistema WaterTank.*

- **B. Implementación de Código:** Firmware C++ (.ino) y clase ArduinoSerial.java en Servicios.
    - *Análisis del Código:* Se programó el firmware para que el Arduino actúe de manera eficiente, midiendo solo bajo demanda al recibir comandos por el puerto COM. En el lado de Java, se implementó el puente de comunicación para capturar las tramas de texto y convertirlas en eventos del sistema.

- **C. Justificación Técnica:**
    - **GRASP (Experto en Información):** Se le asignó a las clases de Servicios la responsabilidad de traducir los bytes de hardware a un formato que el dominio de Java pueda procesar.

Derly Novoa (Lógica de Negocio y Actuadores - Hardware Físico)

Super significa "llama al constructor de la clase que estoy extendiendo

La palabra **super(20.0f, 80.0f);** es literalmente el Hijo llamando al Padre y diciéndole: *"Oye papá, aquí te mando el 20 y el 80 que me pediste para que ambos podamos existir"*.

1. En el **Domain** (El Rey): Solo va la Interfaz (IPump)

El Dominio no se ensucia las manos. Solo dicta el contrato. Él dice: *"No me importa si es una bomba de agua, un motor nuclear o un texto en consola. Yo solo exijo que lo que sea que me pasen, tenga un botón de turnOn() y turnOff()"*.

2. En **Services** (El Mayordomo): Van las implementaciones

Aquí es donde viven las bombas de verdad (o las de mentira). Si esa "Pump normalita" que tienes es la que guarda el boolean isActive y tiene la lógica de encenderse, **debe ir en Services**.

- **A. Componente de Hardware:** Módulo Relé y Mini-bomba de agua (Actuación).
    - *Proceso de Análisis:* Se seleccionó un relé para el control de potencia, asegurando una reacción física observable en el prototipo bajo los estándares de seguridad electrónica de la placa.

- **B. Implementación de Código:** Lógica en WaterLevelManager y definición de la interfaz IPump.
    - *Análisis del Código:* Se encapsuló la regla de negocio (evaluar el nivel crítico del tanque) dentro del método de ejecución del Manager. Este evalúa el porcentaje y dispara la acción (turnOn()) en la interfaz de la bomba, sin acoplarse al código serial directamente.

- **C. Justificación Técnica:**
    - **SOLID (Responsabilidad Única - SRP):** El Manager toma la decisión de encender basándose en las variables, la Bomba solo tiene la responsabilidad de encenderse.
    - **GRASP (Controlador):** El Manager actúa como coordinador del flujo entre lo que se lee y la acción física a ejecutar.

Santiago Chavarro (Persistencia de Datos y Trazabilidad)

- **A. Responsabilidad:** Implementación del almacenamiento del proyecto.

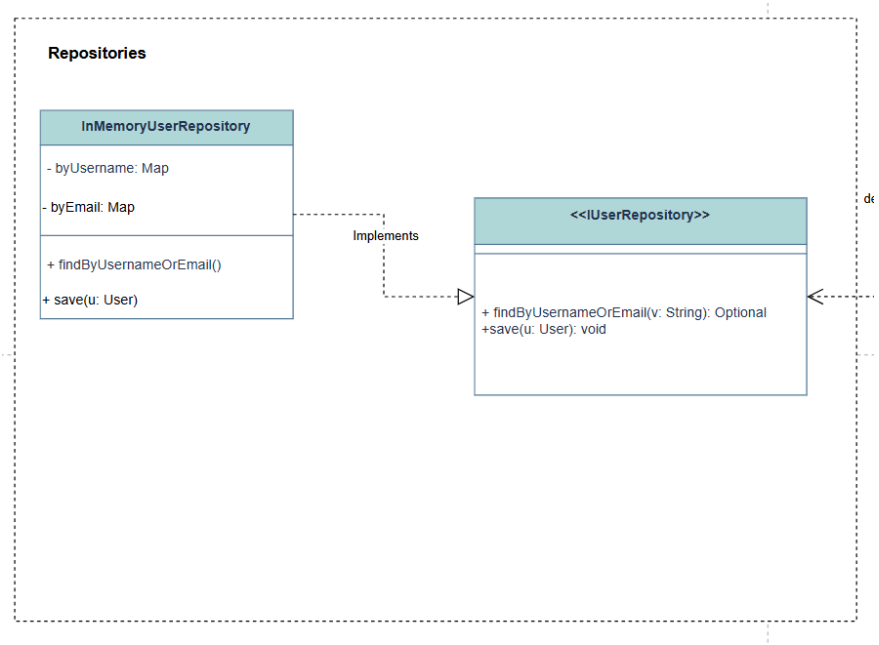
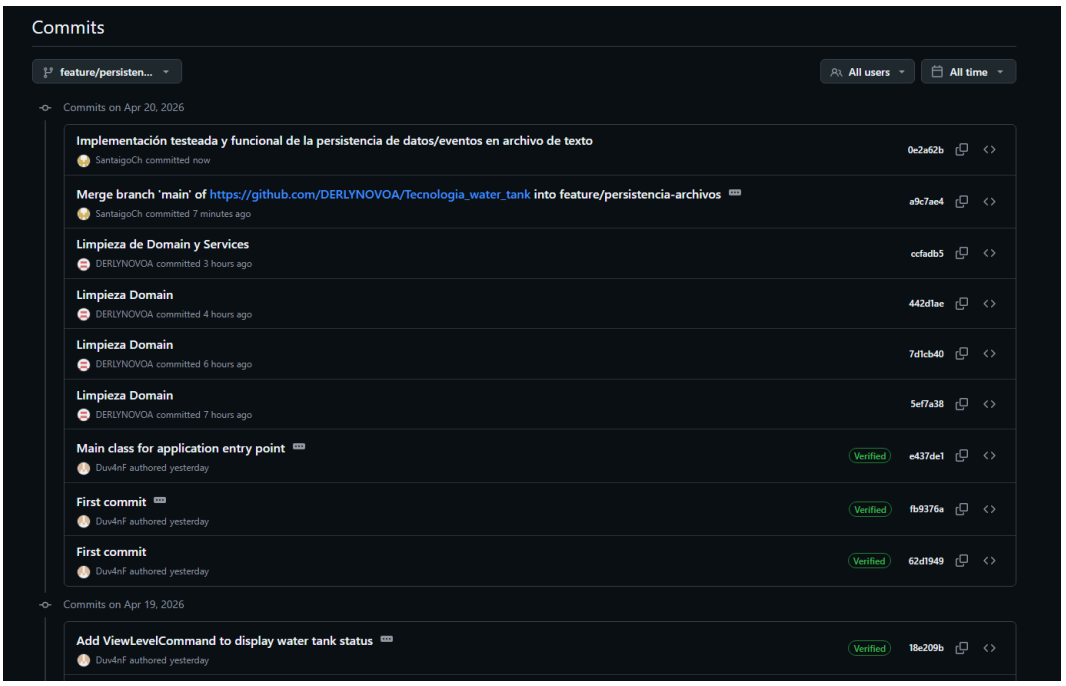


Figura 5. *Conjunto de clases de la capa de Repositorios del sistema WaterTank.*

- **B. Implementación de Código:** Interfaz IRepositoryLog y capa de persistencia mediante archivos.
    - *Análisis del Código:* Se implementó un sistema de registro (logs) usando almacenamiento en archivos de texto plano, guardando los eventos críticos del sistema (ej. niveles de advertencia y encendido de bomba) junto con marcas de tiempo (timestamps).

- **C. Justificación Técnica:**
    - **SOLID (Inversión de Dependencias - DIP):** El sistema principal depende de una abstracción (IRepositoryLog), permitiendo que el almacenamiento sea un archivo .txt hoy, y una base de datos robusta en el futuro, sin reescribir el negocio.

D. Evidencia de colaboración en el repositorio de github:



Duvan Baquero (Interfaz de Usuario y Enrutamiento)

- **A. Responsabilidad:** Sistema visual de Consola, interacciones y el punto de entrada de la aplicación.

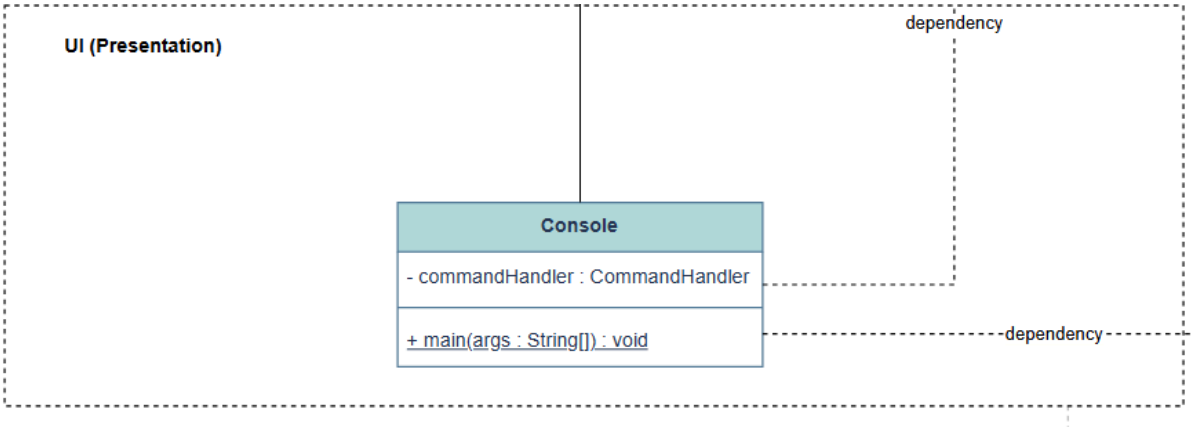


Figura 5. *Conjunto de clases de la capa de presentación del sistema WaterTank.*

- **B. Implementación de Código:** Paquete UI, Console.java y ensamble en el Main.
  - *Análisis del Código:* Se diseñó la interfaz de consola mediante la cual el usuario envía los comandos de inicio. Se estructuró el Main como un inyector

manual de dependencias, instanciando repositorios, servicios y finalmente levantando la interfaz, asegurando que la UI esté desacoplada.

- **C. Justificación Técnica:**
  - **GRASP (Indirección):** El patrón de comandos en la UI sirve como mediador, impidiendo que la vista acceda y modifique el hardware o el dominio directamente.

Desarrollé:

Archivos creados:

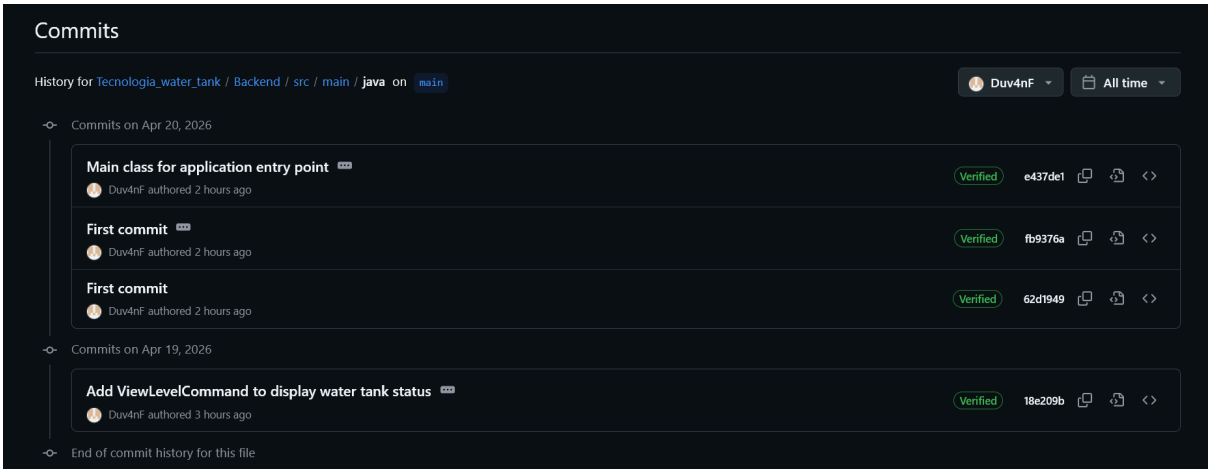
Commands/ViewLevelCommand.java ← nuevo

- Lee el nivel del sensor vía WaterTank
- Muestra nivel en cm, porcentaje y estado de la bomba
- Aplica patrón **Indirección (GRASP)**: la consola nunca toca el dominio directo

Main.java ← en raíz de java/

- Crea sensores → crea WaterTank → registra commands en CommandHandler → inyecta en Console
- **Inyección manual de dependencias**: ninguna clase crea sus propias dependencias

D. Evidencia de colaboración en el repositorio de github:



Diseñé la interfaz de usuario de consola y el CommandHandler aplicando el patrón de Indirección, logrando que la consola nunca toque la lógica del tanque directamente.

3. Matriz de Casos de Uso y Sensores

CASO DE USO	SENSOR (ES)	CONDICIÓN DE ACTIVACIÓN	REACCIÓN ESPERADA	DATOS QUE PERSISTAN	IMPLEMENTADO?
Monitoreo de Nivel	Ultrasónico HC-SR04	Nivel < 20% (umbral mínimo). Nivel > 80% (umbral máximo).	Activar MOSFET para encender la Mini-bomba de agua + Mostrar alerta en consola.	Nivel (%), estado bomba (ON/OFF), timestamp de cada evento	SÍ
Alerta de Temperatura	Sensor DS18B20	Temperatura > 35°C (sobrecalentamiento crítico).	Encender LED rojo + activar alarma sonora + mostrar alerta en consola.	Temperatura (°C), timestamp de alerta, duración de condición anómala.	NO
Vaciado de Emergencia	Comando en Consola	Comando 'DRENAR' del usuario o botón físico presionado.	Abrir válvula de drenaje + detener bomba + confirmar acción en consola.	Acción de usuario, timestamp de inicio, nivel al momento del drenaje.	NO

Casos de Uso

Caso de Uso 1: Monitoreo de Nivel

Estado: Implementado

Sensor/Periférico: Sensor ultrasónico HC-SR04, conectado a los pines TRIG (9) y ECHO (10) del Arduino. Este sensor mide la distancia entre el tope del tanque y la superficie del agua mediante pulsos ultrasónicos, permitiendo calcular el nivel actual expresado en centímetros y en porcentaje de llenado.

Condición de Activación: El sistema evalúa el nivel de forma continua. Se distinguen dos umbrales críticos: si el nivel desciende por debajo del **20%** de la capacidad del tanque (umbral mínimo), el sistema interpreta una condición de escasez; si supera el **80%** (umbral máximo), interpreta una condición de llenado completo.

Reacción Esperada: Al cruzar el umbral mínimo (< 20%), el sistema activa automáticamente la mini-bomba DC mediante el relé de 1 canal conectado al pin 13, encendiéndola para iniciar el llenado. Al alcanzar el umbral máximo (> 80%), la bomba se apaga automáticamente para evitar desbordamiento. En ambos casos, el evento queda

registrado en consola con el nivel en porcentaje, el estado de la bomba (ON/OFF) y el timestamp del momento.

Datos que persisten: Nivel (%), estado de la bomba y timestamp de cada evento.

Caso de Uso 2: Alerta de Temperatura

Estado: No implementado

Sensor/Periférico: Sensor de temperatura DS18B20, sensor digital de precisión que se comunica con el Arduino mediante protocolo 1-Wire. Permite medir la temperatura del agua en grados Celsius con alta exactitud.

Condición de Activación: El sistema monitorea continuamente la temperatura del agua. La condición se activa cuando la temperatura supera los **35 °C**, valor que indica un sobrecalentamiento crítico que puede comprometer la calidad del agua o el funcionamiento del sistema.

Reacción Esperada: Al detectar la condición anómala, el sistema enciende un **LED rojo** como indicador visual de alerta, activa una **alarma sonora** como señal auditiva, y muestra un mensaje de advertencia en consola informando la temperatura registrada y el momento de la alerta. El sistema también registra la duración durante la cual se mantuvo la condición de sobrecalentamiento.

Datos que persisten: Temperatura (°C), timestamp de inicio de la alerta y duración de la condición anómala.

Caso de Uso 3: Vaciado de Emergencia

Estado: No implementado

Sensor/Periférico: No requiere sensor físico de medición; se activa mediante **comando de texto en consola** (DRENAR) o mediante un **botón físico** conectado directamente al Arduino. Ambos mecanismos representan una acción manual deliberada del operador.

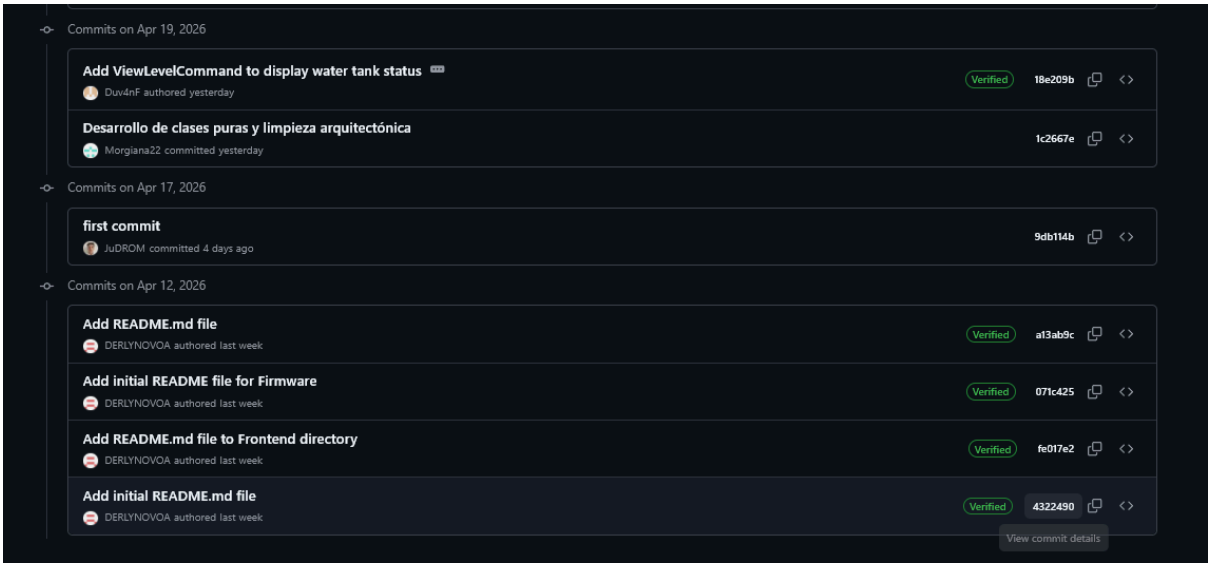
Condición de Activación: El sistema reconoce la instrucción de vaciado cuando el usuario ingresa el comando DRENAR desde la interfaz de consola o cuando presiona el botón físico habilitado para emergencias. Esta acción tiene prioridad sobre cualquier lógica automática en curso.

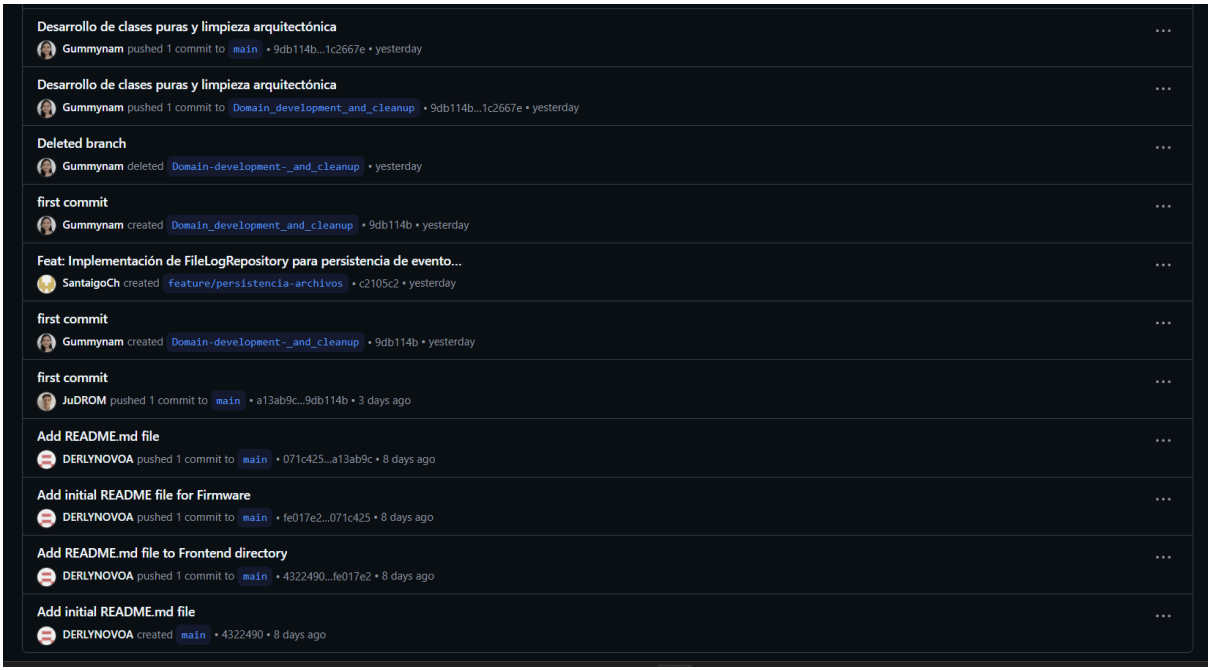
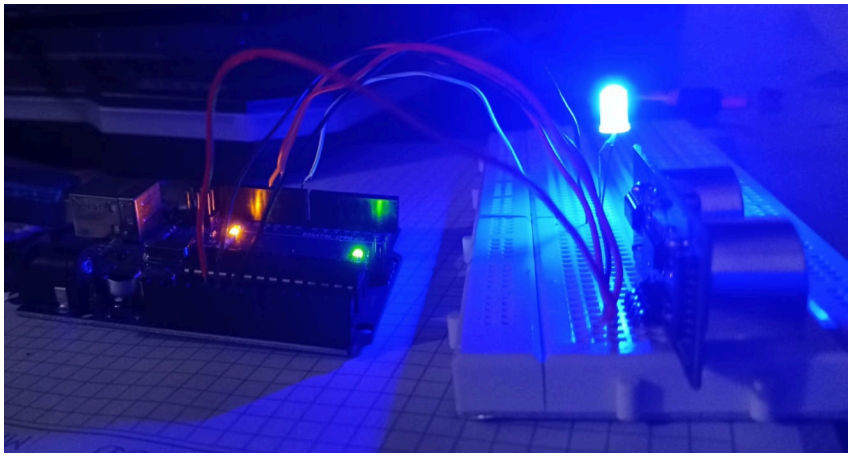
Reacción Esperada: Al recibir la instrucción, el sistema detiene inmediatamente la bomba (si estuviera activa), abre la válvula de drenaje para permitir la salida del agua, y muestra en consola una confirmación de la acción ejecutada junto con el nivel del tanque al momento de iniciar el vaciado. Esto garantiza trazabilidad completa de la intervención de emergencia.

Datos que persisten: Acción del usuario, timestamp de inicio del vaciado y nivel del tanque en el momento del drenaje.

Evidencias de Ejecución

- **Historial del Repositorio:** *(Capturas de las ramas y commits de Derly, Astrid, Juan, Santiago y Duvan en GitHub).*





- **Demostración Funcional:**